

# Chapter 5

## Sequential Logic

### 5.1 Objectives

1. Understand timing diagrams, gate delays and critical path
2. Design Hazard-free two level circuits
3. Building blocks of sequential circuits
4. Analyze a sequential circuit and derive a state-table and a state-graph
5. Derive a state graph or state table from a word description of the problem
6. Understanding the structure of an FPGA

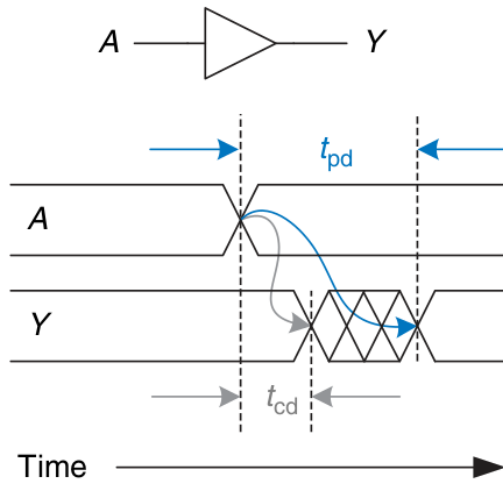
### 5.2 Why do we need sequential circuits?

**Example 5.1.** *Think about this problem: Design an occupancy counter that depends on a sensor  $S$  at the class door. The sensor is triggered every time a person passes through the door. The counter can be reset to zero with a reset button. Assume we only need up to two bit counter  $C_1C_0$ . Draw a truth table for this circuit. Do you have requisite knowledge for designing this circuit? Can this circuit be designed without a memory element?*

### 5.3 Timing diagrams and propagation delays

**Example 5.2** (Timing diagram). *Draw a timing diagram for an ideal NAND gate.*

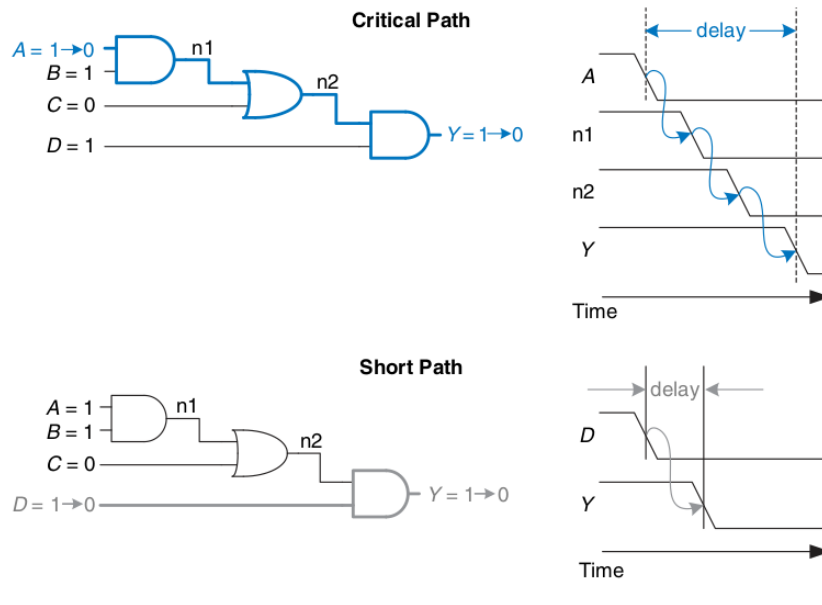
## 5.3.1 Delays



**Definition 5.1** (Propagation delay ( $t_{pd}$ )).

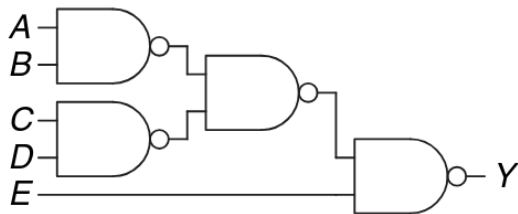
**Definition 5.2** (Contamination delay ( $t_{cd}$ )).

## 5.3.2 Paths



**Example 5.3.** Find the propagation delay of the circuit above given that propagation delay of each gate is 100ps add contamination delay of 60ps.

**Example 5.4** (Ex 2.43 [1]). Determine the propagation delay and contamination delay of the circuit in Figure 5.1. Use the gate delays given in Table 5.2.



**Figure 2.83** Circuit schematic

Figure 5.1: Circuit

Table 2.8 Gate delays for Exercises 2.43–2.47

| Gate         | $t_{pd}$ (ps) | $t_{cd}$ (ps) |
|--------------|---------------|---------------|
| NOT          | 15            | 10            |
| 2-input NAND | 20            | 15            |
| 3-input NAND | 30            | 25            |
| 2-input NOR  | 30            | 25            |
| 3-input NOR  | 45            | 35            |
| 2-input AND  | 30            | 25            |
| 3-input AND  | 40            | 30            |
| 2-input OR   | 40            | 30            |
| 3-input OR   | 55            | 45            |
| 2-input XOR  | 60            | 40            |

Figure 5.2: Delays

**Problem 5.1** (10 marks, Ex 2.44 [1]). *Determine the propagation delay and contamination delay of the circuit in Figure 5.3. Use the gate delays given in Figure 5.2.*

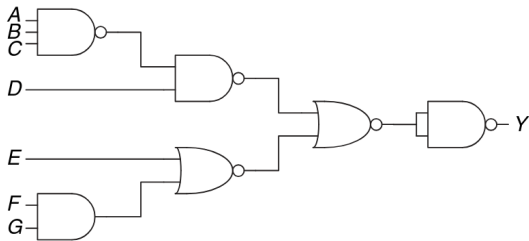
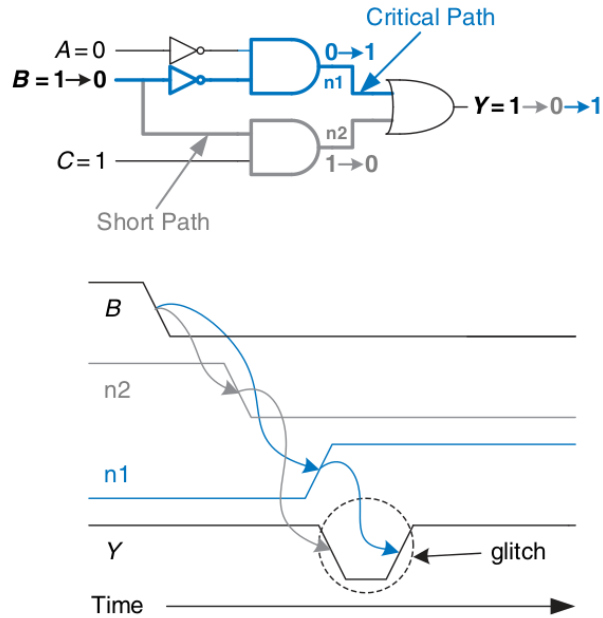


Figure 2.84 Circuit schematic

Figure 5.3: Circuit

## 5.4 Glitches or Hazards



**Definition 5.3** (Glitch or Hazard).

**Example 5.5.** Design a circuit that fixes the glitch in the above circuit (also known as *glitch-free* or *hazard-free* circuit).

**Problem 5.2.** Find a minimal Boolean equation for the function in Figure 2.85. Remember to take advantage of the don't care entries (marked X) (10 marks).

| A | B | C | D | Y |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | X |
| 0 | 0 | 0 | 1 | X |
| 0 | 0 | 1 | 0 | X |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | X |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | X |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | X |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | X |
| 1 | 1 | 1 | 1 | 1 |

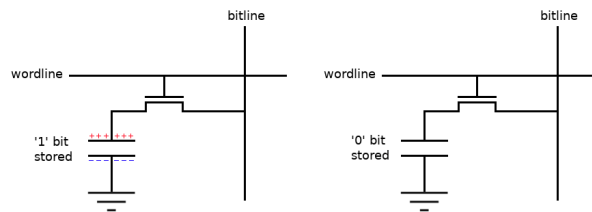
Figure 2.85 Truth table for Exercise 2.28

1. Sketch a circuit for the function (10 marks).
2. Does your circuit from have any potential glitches when one of the inputs changes? If not, explain why not. If so, show how to modify the circuit to eliminate the glitches (10 marks).

## 5.5 How to create memory element from circuits

Two types of memory

1. Volatile memory. For example, RAM, CPU registers.
2. Non-volatile memory. For example, SSD, Flash drives. (Not covered in this course)
  - (a) Memories that require periodic refreshing. For example, DRAM: Dynamics Random Access memory (Not covered in this course)



1

- (b) Memories that are always refreshing. For example, SRAM: Static Random Access memory [3, Appendix B.64]

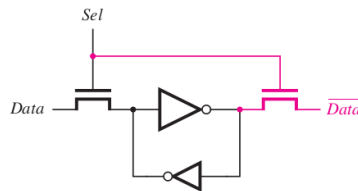
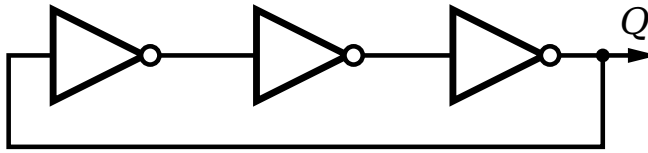


Figure B.64 An SRAM cell.

<sup>1</sup>Image source: [allaboutcircuits.com/technical-articles/introduction-to-dram-dynamic-random-access-memory/](http://allaboutcircuits.com/technical-articles/introduction-to-dram-dynamic-random-access-memory/)

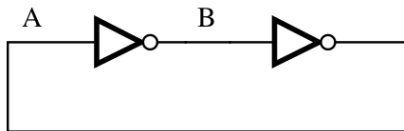
## 5.6 Latches and Flip-Flops [1, Sec 3.2]

**Example 5.6** (Ring oscillator ). [1, Sec 3.31] How many stable states does the following circuit have?



**Definition 5.4** (Astable circuits).

**Example 5.7.** Analyze the timing diagram of the following circuit.

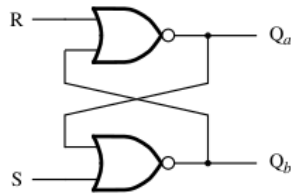


**Definition 5.5** (Bistable circuits).

**Definition 5.6** (Characteristic or state table). Draw the characteristic or state table of the above circuit.

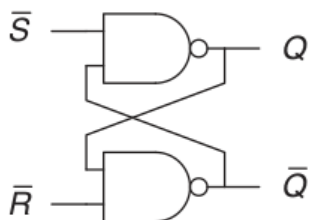
### 5.6.1 SR (Set-Reset) latch [1, Sec 3.2.1]

**Definition 5.7** (SR latch). *The following circuit is called the SR latch.*



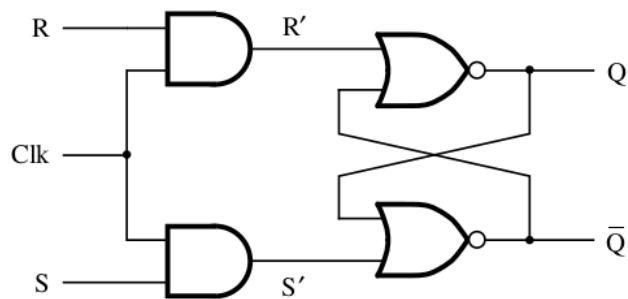
1. How many stable states does this circuit have?
2. Draw its characteristic or state table.
3. Draw SR latch symbol

**Problem 5.3** (SR latch using NAND gates). *Draw the characteristic or state table for the following circuit*



### 5.6.2 Gated SR latch [3, Sec 5.2]

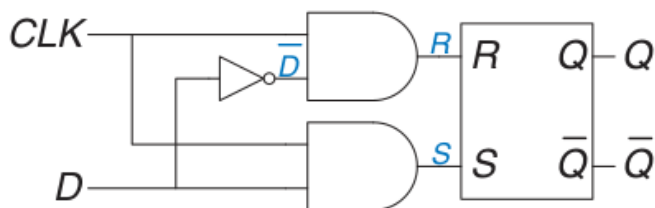
**Definition 5.8** (Gated SR latch). *The following circuit is called the Gated SR latch.*



1. Draw its characteristic table.
2. Draw the Gated SR latch symbol

### 5.6.3 D (Data) latch [1, Sec 3.2.2]

**Definition 5.9** (D latch). *The following circuit is called the D latch.*



1. Draw its characteristic table.
2. Draw the D latch symbol

**Problem 5.4.** Consider the timing diagram in Figure 5.4. Assuming that the  $D$  and Clock inputs shown are applied to the circuit in Figure 5.5, draw waveforms for the  $Q_a$ ,  $Q_b$ , and  $Q_c$  signals. (10 marks) [?, Prob 5.1]

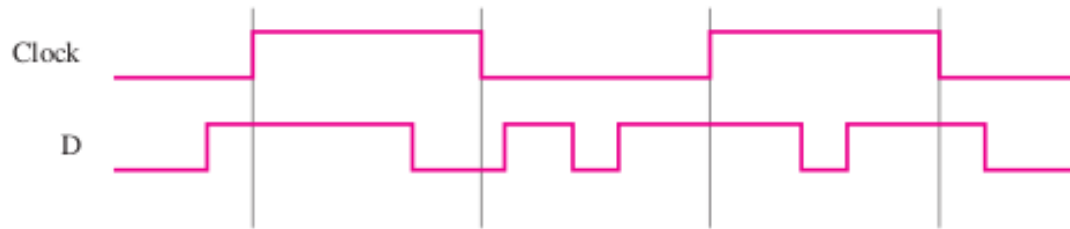
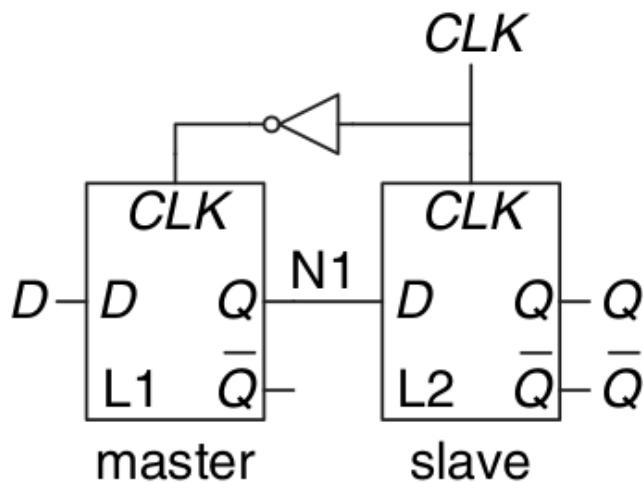


Figure 5.4: Inputs for Prob 5.4

#### 5.6.4 D flip-flop [1, Sec 3.2.2]

**Definition 5.10** (D flip-flop). The following circuit is called the D flip-flop.



1. Draw its timing diagram
2. Draw its characteristic table.
3. Draw the D flip-flop symbol

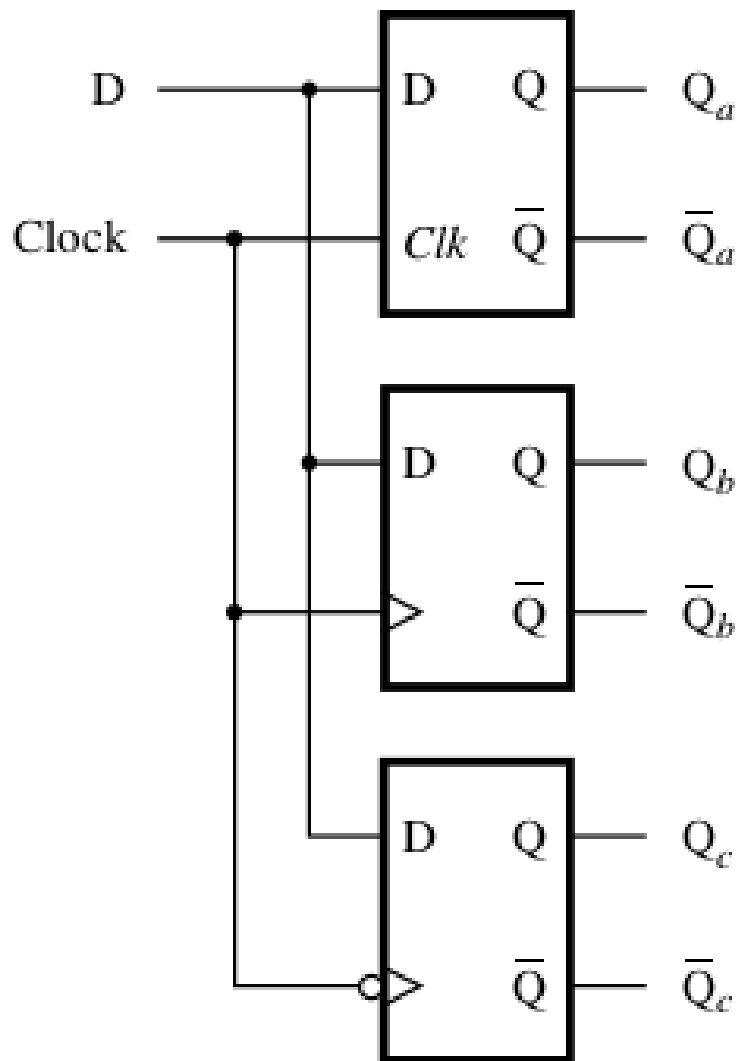


Figure 5.5: Circuit for Prob 5.4. Recall that  $Clk$  with a “▷” symbol indicates rising-edge (positive-edge) triggered flip-flop.  $Clk$  without “▷” symbol indicates a level-triggered latch.  $Clk$  with “◁” symbol indicates a falling-edge (negative-edge) triggered flip-flop.

**Remark 5.1.** What is the difference between a latch and a flip-flop?

**Example 5.8.** Add a RESET signal to the D flip-flop that resets the state of flip-flop to 0.

**Example 5.9.** The toggle (T) flip-flop has one input, CLK, and one output, Q. On each rising edge of CLK, Q toggles to the complement of its previous value. Draw a schematic for a T flip-flop using a D flip-flop and an inverter.

**Problem 5.5.** A JK flip-flop receives a clock and two inputs, J and K. On the rising edge of the clock, it updates the output, Q. If J and K are both 0, Q retains its old value. If only J is 1, Q becomes 1. If only K is 1, Q becomes 0. If both J and K are 1, Q becomes the opposite of its present state.

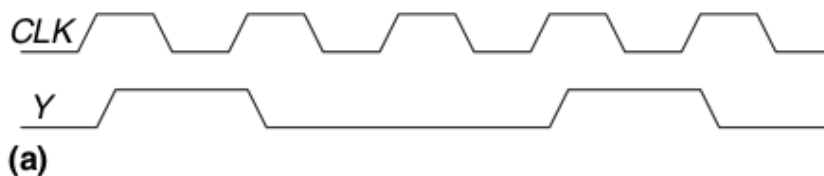
1. Construct a JK flip-flop using a D flip-flop and some combinational logic.
2. Construct a D flip-flop using a JK flip-flop and some combinational logic.
3. Construct a T flip-flop (see Exercise 3.9) using a JK flip-flop.

## 5.7 Finite State Machines [1, Sec 3.4]

2

**Example 5.10.** Design an occupancy counter that depends on a sensor S at the class door. The sensor is triggered every time a person passes through the door. Assume that the counter starts at zero. Assume we only need up to two bit counter  $C_1C_0$ . Draw a state table for this circuit.

**Problem 5.6.** A divide-by-N counter has one output and no inputs. The output Y is HIGH for one clock cycle out of every N. In other words, the output divides the frequency of the clock by N. The waveform for a divide-by-3 counter is shown here:



Sketch circuit designs for such a counter

---

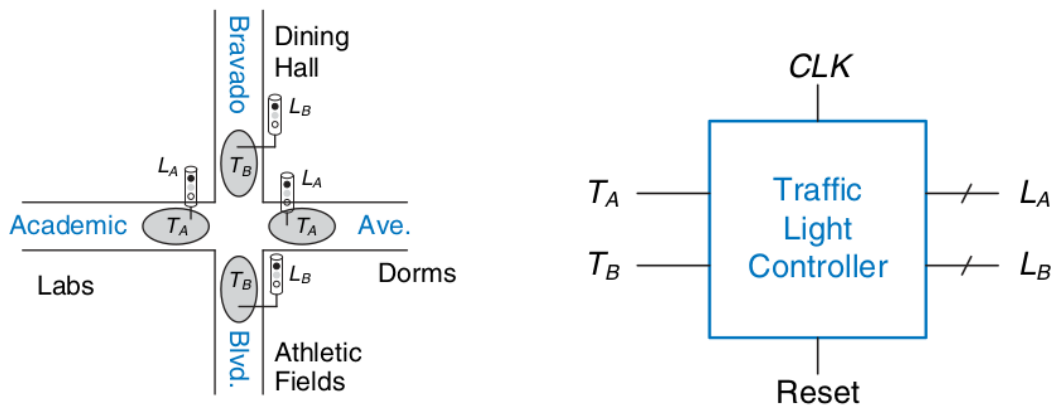
<sup>2</sup>These notes will not fit on your note sheet.

**Problem 5.7.** Design a 3-bit counter which counts in the sequence: 001, 011, 010, 110, 111, 100, (repeat) 001, ...

**Example 5.11.** Design an odd-even counter for an single bit input. The output of this circuit should be 1 if the number of 1s to the input have been odd so far and 0 otherwise.

**Example 5.12** (Sequence detectors). A sequential circuit has one input and one output. The output becomes 1 and remain 1 thereafter when at least two 0's and at least two 1's have occurred as inputs regardless of the order of

**Example 5.13.** Consider the problem of inventing a controller for a traffic light at a busy intersection on campus. There are two traffic sensors,  $T_A$  and  $T_B$ , on Academic Ave. and Bravado Blvd., respectively. Each sensor indicates TRUE if students are present and FALSE if the street is empty. There are two traffic lights,  $L_A$  and  $L_B$ , to control traffic. Each light receives digital inputs specifying whether it should be green, yellow, or red. When the system is reset, the lights are green on Academic Ave. and red on Bravado Blvd. As long as traffic is present on Academic Ave., the lights do not change. When there is no longer traffic on Academic Ave., the light on Academic Ave. becomes yellow for 5 seconds before it turns red and Bravado Blvd.'s light turns green. Similarly, the Bravado Blvd. light remains green as long as traffic is present on the boulevard, then turns yellow and eventually red.



1. Draw a state transition diagram
2. Draw a state table
3. Assign binary encodings to each of the states
4. Redraw the state table with binary encodings. Design a minimal SOP boolean expression.
5. Assign binary encodings to each of the output and redraw the output table. Design a minimal SOP boolean expression for the outputs.

**Problem 5.8.** Design a circuit for a 2x2 pixel resolution pong game, where the ball can only occupy 4 possible pixels and a single paddle occupies another 2 pixels. The ball bounces off the paddle when the paddle is in the correct row. To keep it interesting, the ball takes a different path from the source path. Track the score with a single bit counter.